Principles of Computer Programming (2440099) — Week 2 Lecture 1 Handout

How Values Are Actually Stored: the Fundamental Types

Keep this. You will want the type-size table and the signed/unsigned distinction constantly from here on — this course does not re-teach fundamental types once the semester's projects start.

1. Key terms

Term	Definition
Bit	The smallest unit of memory: a single 0 or 1.
Byte	8 bits, the smallest unit memory can address individually.
sizeof	An operator that reports how many bytes a type or variable occupies.
Signed / unsigned	Signed types can hold negative numbers; unsigned types cannot, and use that traded-away range for larger positive values instead.
Fixed-width type	A type from <cstdint> (e.g. int32_t) whose size is exactly guaranteed, on every machine — unlike int or long.
Variable	A named, typed location in memory whose value can change.
Constant (const)	A named value, fixed at initialization; reassigning it afterward is a compile error.
Expression	Any combination of literals, variables, and operators that the compiler can evaluate down to a single value.
Initialization	Giving a variable its first value at the moment it's declared, instead of leaving it garbage.
Reference	Another name for an already-existing variable, bound at declaration and never rebound afterward.
Literal	A value written directly into source code (42, 3.14, 'A', "hi", true), rather than read from a variable.
Lvalue	A value that names a persistent, addressable location (a variable) — can sit on the left of =.
Rvalue	A temporary value with no address of its own (a literal, or most calculation results) — cannot sit on the left of =.
Precedence	Which operator in an expression runs first when more than one is present.
Associativity	The direction ties between same-precedence operators break — left-to-right for arithmetic, right-to-left for assignment.
Sequence point	A point in a program (roughly, a statement boundary) after which all earlier side effects are guaranteed complete.

2. Variables, constants, and references

A variable is declared with a type and a name, and should always be initialized at the same time — an uninitialized variable holds whatever bits were already in that memory, not zero.

```
int score;                    // declared -- UNINITIALIZED (garbage)
int score = 0;                // declared AND initialized -- do this, always
const double PI = 3.14159;    // a constant -- cannot be reassigned
```

A name (identifier) is letters, digits, and underscores, never starting with a digit, and case-sensitive. This course writes variable names in lowerCamelCase. const locks a value in at initialization; reassigning a const afterward is a compile error.

A reference is another name for an already-existing variable — not a copy, and not a new variable:

```
int x = 5;
int& ref = x;   // ref is just another name for x
ref = 10;
std::cout << x;   // 10 -- changing ref changed x
```

A reference must be bound to a variable the instant it's declared, and can never be rebound afterward. The & here sits in the TYPE (int&) and means ‘reference to’ — a different job from the address-of & you'll meet in week 7.

3. Lvalue and rvalue

x is an lvalue — it names a specific, addressable memory location. 5 is an rvalue — a temporary value with no address of its own.

```
int x = 5;    // x is an lvalue -- it has an address
x = 10;       // fine: assigning INTO an lvalue
// 5 = x;     // ERROR: 5 is an rvalue -- nothing to assign into
```

> **KEY LINE** Rule of thumb: if you could take its address (&x), it's an lvalue. This is exactly why references only bind to lvalues, and why the left side of = must be one too.

4. Literals

Every number written directly into code is a literal, and C++ has more than one notation for it:

```
int decimal = 42;
int octal   = 052;    // a LEADING ZERO -- octal, also 42
int hex     = 0x2A;   // a leading 0x -- hexadecimal, also 42
```

> **KEY LINE** A leading 0 on an integer literal means OCTAL, not decimal — 052 is 42, not fifty-two. Never pad an integer literal with a leading zero by accident.

A numeric literal has a default type (a plain whole number is int, a plain decimal like 3.14 is double); a suffix overrides it:

```
long      a = 100L;    // force long
unsigned  b = 100U;    // force unsigned
long long c = 100LL;   // force long long
float     d = 3.14f;   // force float -- plain 3.14 would be double
```

> **KEY LINE** Every floating-point literal is a double, by default — 3.14, .5, 2.5e3, all of them. The ONLY way to get a float is the f suffix. This is exactly why this course's convention is double everywhere.

Decimal literals have more than one written form:

```
double a = 3.14;
double b = .5;      // same as 0.5
double c = 5.;       // same as 5.0
double d = 2.5e3;    // scientific notation -- 2.5 x 10^3 -- 2500
```

5. Operators

Arithmetic operators:

```
+   addition          a + b
-   subtraction       a - b
*   multiplication    a * b
/   division          a / b   (integer division truncates -- next lecture)
%   modulo            a % b   (the remainder)
```

= is its own operator — like every operator, it evaluates to a value (the one just assigned), which is what makes chained assignment legal:

```
int a, b, c;
a = b = c = 5;   // right to left: c=5, then b=(c=5), then a=(b=5)
std::cout << a << b << c;   // 555
```

Assignment operators — every compound form does a read, the operation, and a write-back in one step:

```
=    assign               total = 5;
+=   add and assign       total += 5;    // same as total = total + 5;
-=   subtract and assign  total -= 5;
*=   multiply and assign  total *= 2;
/=   divide and assign    total /= 2;
%=   modulo and assign    total %= 2;
++   increment (add 1)    ++count;
--   decrement (sub 1)    --count;
```

> **KEY LINE** = assigns a value; it is NOT the same as mathematical equality. The compiler will not stop you from writing if (x = 3) by accident — week 3 covers the bug that causes, in depth.

++/-- have a prefix and a postfix form — both change the variable the same way, but the EXPRESSION they form evaluates differently:

```
int i = 5;
std::cout << i++;   // prints 5 (OLD value), THEN i becomes 6
int j = 5;
std::cout << ++j;   // j becomes 6, THEN prints 6 (NEW value)
```

One more operator, rare but real: the comma operator evaluates its left side, discards the result, then evaluates and returns its right side:

```
int a = (1, 2, 3);        // evaluates 1, then 2, then 3 -- a is 3
int x = 0;
int y = (++x, x + 10);    // x becomes 1, then y = 1 + 10 -- y is 11
```

> **KEY LINE** The comma in int x = 0, y = 1; is NOT this operator — that's just declaration-list syntax. The real comma operator only exists inside an expression, usually in parentheses.

6. What is an expression?

Now, precisely: an expression is any combination of literals, variables, and operators that the compiler can evaluate down to a single value. 7 is an expression; score is an expression; score + 7 is an expression. Even score = 7 is an expression — assignment evaluates to the value it just assigned.

7. Operator precedence and associativity

Precedence decides which operator runs first; associativity breaks ties between operators of the same precedence.

```
*  /  %              (highest, so far)
+  -
=  +=  -=  *=  /=  %=   (right-to-left)
,                     (lowest, so far -- more next lecture)


std::cout << 2 + 3 * 4;     // 14 -- * before +
std::cout << 20 - 5 - 3;    // 12 -- left to right
int a = b = 5;              // right to left
```

8. Sequence points

Between sequence points, the compiler may evaluate an expression's parts in any order. Modifying the same variable twice with no sequence point in between is undefined behavior:

```
int i = 0;
i = i++ + 1;   // UNDEFINED BEHAVIOR -- never write this
```

> **KEY LINE** Never modify the same variable more than once in an expression without a sequence point (a semicolon, &&/||, a function-call boundary) between the modifications.

9. Reading and writing with streams

std::cout and std::cin use operators too: << (stream insertion) sends a value to a stream; >> (stream extraction) reads one out. Both chain:

```
int age;
std::cout << "Enter your age: ";
std::cin >> age;
std::cout << "You are " << age << " years old.\n";
```

std::cout << a << b; with a = 3 and b = 4 prints 34, not 7 — << sends each value to the stream as text, one after another; it never adds them.

10. The fundamental types

“Typical” below is what you'll actually see on this course's WSL2/Debian machines. “Guaranteed” is the only part the C++ standard actually promises — memorize the ORDER, not the numbers.

Every integer type in the table also has an unsigned form (unsigned int, unsigned long, ...): same size, no negative numbers, roughly double the positive range in exchange.

11. Why this course defaults to double

float and double both store approximate, not exact, real numbers (the full story is next lecture). float simply stores a shorter approximation — it uses less memory but loses precision faster. Mixing float and double in one expression silently converts one of them, which is one more thing to get subtly wrong for very little benefit on modern hardware. Rule for this semester: use double, everywhere, unless a task specifically tells you otherwise.

12. Fixed-width types — when “typical” isn't good enough

int, long, and friends promise an ORDER of sizes, not an exact one. When a task needs an exact size regardless of machine — talking to hardware, reading a file format someone else defined, or precisely counting bytes — include <cstdint> instead.

```
#include <cstdint>

int8_t   a;   //  8 bits, signed,   always
uint8_t  b;   //  8 bits, unsigned, always
int32_t  c;   // 32 bits, signed,   always
uint32_t d;   // 32 bits, unsigned, always
int64_t  e;   // 64 bits, signed,   always
uint64_t f;   // 64 bits, unsigned, always
```

Read the name left to right: (u)int, the bit width, _t. uint8_t is “unsigned integer, 8 bits, a type.”

> **TODAY'S LAB** Micro-exercise 1 (next lecture) has you print these sizes yourself, trigger an overflow on purpose, and compare two doubles the wrong way and the right way. Nothing here is graded today — it's the vocabulary for that exercise and for everything built on top of it this semester.

Type	Typical size	Guaranteed	Used for
bool	1 byte	at least 1 byte	true / false
char	1 byte	at least 1 byte	a small integer; conventionally a character code
short	2 bytes	≥ char	a small integer, rarely used by name in this course
int	4 bytes	≥ short	the default integer type — use this unless you have a reason not to
long	8 bytes	≥ int	a larger integer, when int's range genuinely isn't enough
long long	8 bytes	≥ long	guaranteed at least 64 bits — the safest choice for “definitely big enough”
float	4 bytes	smaller than double	rarely used this semester — see §11
double	8 bytes	≥ float	this course's default for every non-integer number